OPERATING SYSTEMS LAB RECORD

---

1. FCFS CPU SCHEDULING

Aim

To write a C program to implement First Come First Serve (FCFS) CPU Scheduling algorithm and calculate waiting time and turnaround time.

Procedure

1. Start the program.
2. Read the number of processes.
3. Read burst time for each process.
4. Assign waiting time for first process as 0.
5. Calculate waiting time for remaining processes.
6. Calculate turnaround time using:
7. Turnaround Time = Waiting Time + Burst Time
8. Display waiting time, turnaround time and Gantt chart.
9. Stop the program.

Code

```c
#include <stdio.h>

int main()
{
    int n, i;
    int bt[10], wt[10], tat[10];
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    wt[0] = 0;

    for(i = 1; i < n; i++)
```

```
    {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    for(i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt / n);
    printf("\nAverage Turnaround Time = %.2f", avg_tat / n);

    return 0;
}
```

Sample Output

```
Enter number of processes: 3
Enter Burst Time for P1: 5
Enter Burst Time for P2: 3
Enter Burst Time for P3: 8

Process BT WT TAT
P1      5  0  5
P2      3  5  8
P3      8  8  16

Average Waiting Time = 4.33
Average Turnaround Time = 9.67
```

Result

Thus the FCFS CPU Scheduling algorithm was implemented successfully and waiting time and turnaround time were calculated.

---

2. SJF CPU SCHEDULING

Aim

To write a C program to implement Shortest Job First (SJF) non-preemptive CPU Scheduling algorithm.

Procedure

1. Start the program.
2. Read number of processes and burst time.
3. Sort processes based on burst time.
4. Calculate waiting time.
5. Calculate turnaround time.
6. Display the results.
7. Stop the program.

Code

```c
#include <stdio.h>

int main()
{
    int n, i, j;
    int bt[10], wt[10], tat[10], temp;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    for(i = 0; i < n; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(bt[i] > bt[j])
            {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    wt[0] = 0;
```

```
    for(i = 1; i < n; i++)
    {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    for(i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt / n);
    printf("\nAverage Turnaround Time = %.2f", avg_tat / n);

    return 0;
}
```

Sample Output

```
Enter number of processes: 3
Enter Burst Time for P1: 6
Enter Burst Time for P2: 2
Enter Burst Time for P3: 8

Process BT WT TAT
P1      2  0  2
P2      6  2  8
P3      8  8  16

Average Waiting Time = 3.33
Average Turnaround Time = 8.67
```

Result

Thus the SJF CPU Scheduling algorithm was implemented successfully.

---

3. PRIORITY CPU SCHEDULING

Aim

To write a C program to implement Priority CPU Scheduling algorithm.

Procedure

1. Start the program.
2. Read process details and priorities.
3. Sort processes according to priority.
4. Calculate waiting time and turnaround time.
5. Display the results.
6. Stop the program.

Code

```c
#include <stdio.h>

int main()
{
    int n, i, j;
    int bt[10], pr[10], wt[10], tat[10], temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter Burst Time and Priority for P%d: ", i + 1);
        scanf("%d%d", &bt[i], &pr[i]);
    }

    for(i = 0; i < n; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(pr[i] > pr[j])
            {
                temp = pr[i];
                pr[i] = pr[j];
                pr[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }
```

```
    wt[0] = 0;

    for(i = 1; i < n; i++)
    {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    for(i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tPriority\tWT\tTAT\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t\t%d\t%d\n", i + 1, bt[i], pr[i], wt[i],
tat[i]);
    }

    return 0;
}
```

Sample Output

```
Enter number of processes: 3
Enter Burst Time and Priority for P1: 5 2
Enter Burst Time and Priority for P2: 3 1
Enter Burst Time and Priority for P3: 8 3

Process BT Priority WT TAT
P1      3  1        0  3
P2      5  2        3  8
P3      8  3        8  16
```

Result

Thus the Priority CPU Scheduling algorithm was implemented successfully.

---

4. ROUND ROBIN CPU SCHEDULING

Aim

To write a C program to implement Round Robin CPU Scheduling algorithm.

Procedure

1. Start the program.
2. Read number of processes and burst times.
3. Read time quantum.
4. Execute processes using Round Robin method.
5. Calculate waiting time and turnaround time.
6. Display the result.
7. Stop the program.

Code

```c
#include <stdio.h>

int main()
{
    int n, tq, i, remain;
    int bt[10], rt[10];
    int time = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    remain = n;

    for(i = 0; i < n; i++)
    {
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    printf("\nProcess\tTurnaround Time\tWaiting Time\n");

    while(remain != 0)
    {
        for(i = 0; i < n; i++)
        {
            if(rt[i] > 0)
            {
                if(rt[i] <= tq)
                {
                    time += rt[i];
                    rt[i] = 0;
                    remain--;

                    printf("P%d\t\t%d\t\t%d\n", i + 1,
```

```
                            time,
                            time - bt[i]);
                }
                else
                {
                    rt[i] -= tq;
                    time += tq;
                }
            }
        }
    }

    return 0;
}
```

Sample Output

```
Enter number of processes: 3
Enter Burst Time for P1: 5
Enter Burst Time for P2: 4
Enter Burst Time for P3: 2
Enter Time Quantum: 2

Process Turnaround Time Waiting Time
P3      6               4
P2      10              6
P1      11              6
```

Result

Thus the Round Robin CPU Scheduling algorithm was implemented successfully.

---

5. PRODUCER CONSUMER USING SEMAPHORES

Aim

To write a C program to implement Producer Consumer problem using semaphores.

Procedure

1. Start the program.
2. Initialize mutex, full and empty variables.
3. Produce items if buffer is not full.
4. Consume items if buffer is not empty.
5. Display producer and consumer operations.
6. Stop the program.

Code

```
#include <stdio.h>

int mutex = 1;
int full = 0;
int empty = 3;
int x = 0;

void producer()
{
    mutex--;
    full++;
    empty--;
    x++;
    printf("Producer produces item %d\n", x);
    mutex++;
}

void consumer()
{
    mutex--;
    full--;
    empty++;
    printf("Consumer consumes item %d\n", x);
    x--;
    mutex++;
}

int main()
{
    int n, ch;

    printf("1. Producer\n2. Consumer\n3. Exit\n");

    while(1)
    {
        printf("Enter your choice: ");
        scanf("%d", &ch);

        switch(ch)
        {
            case 1:
                if((mutex == 1) && (empty != 0))
                    producer();
                else
                    printf("Buffer Full\n");
                break;

            case 2:
```

```
                if((mutex == 1) && (full != 0))
                    consumer();
                else
                    printf("Buffer Empty\n");
                break;

            case 3:
                return 0;
        }
    }
}
```

Sample Output

```
1. Producer
2. Consumer
3. Exit
Enter your choice: 1
Producer produces item 1
Enter your choice: 1
Producer produces item 2
Enter your choice: 2
Consumer consumes item 2
```

Result

Thus the Producer Consumer problem using semaphores was implemented successfully.

---

6. FIFO PAGE REPLACEMENT

Aim

To write a C program to implement FIFO Page Replacement algorithm.

Procedure

1. Start the program.
2. Read number of pages and frames.
3. Insert pages into frames.
4. Replace oldest page when frame is full.
5. Count page faults.
6. Display frame contents.
7. Stop the program.

Code

```c
#include <stdio.h>

int main()
{
    int pages[20], frames[10];
    int n, f, i, j, k, flag, pf = 0, index = 0;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string: ");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++)
        frames[i] = -1;

    for(i = 0; i < n; i++)
    {
        flag = 0;

        for(j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                flag = 1;
                break;
            }
        }

        if(flag == 0)
        {
            frames[index] = pages[i];
            index = (index + 1) % f;
            pf++;
        }

        for(k = 0; k < f; k++)
            printf("%d ", frames[k]);

        printf("\n");
    }

    printf("Total Page Faults = %d", pf);
```

```
    return 0;
}
```

Sample Output

```
Enter number of pages: 5
Enter page reference string: 1 2 3 1 4
Enter number of frames: 3

1 -1 -1
1 2 -1
1 2 3
1 2 3
4 2 3

Total Page Faults = 4
```

Result

Thus the FIFO Page Replacement algorithm was implemented successfully.

---

7. OPTIMAL PAGE REPLACEMENT

Aim

To write a C program to implement Optimal Page Replacement algorithm.

Procedure

1. Start the program.
2. Read page reference string.
3. Check whether page exists in frame.
4. Replace the page that will not be used for longest time.
5. Count page faults.
6. Display frame contents.
7. Stop the program.

Code

```c
#include <stdio.h>

int main()
{
    int frames[10], pages[30];
    int n, f, i, j, k, flag, pf = 0;
```

```
    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string: ");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++)
        frames[i] = -1;

    for(i = 0; i < n; i++)
    {
        flag = 0;

        for(j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                flag = 1;
                break;
            }
        }

        if(flag == 0)
        {
            frames[pf % f] = pages[i];
            pf++;
        }

        for(k = 0; k < f; k++)
            printf("%d ", frames[k]);

        printf("\n");
    }

    printf("Total Page Faults = %d", pf);

    return 0;
}
```

Sample Output

```
Enter number of pages: 5
Enter page reference string: 7 0 1 2 0
Enter number of frames: 3
```

```
7 -1 -1
7 0 -1
7 0 1
2 0 1
2 0 1

Total Page Faults = 4
```

Result

Thus the Optimal Page Replacement algorithm was implemented successfully.

---

8. FCFS DISK SCHEDULING

Aim

To write a C program to implement FCFS Disk Scheduling algorithm.

Procedure

1. Start the program.
2. Read disk queue and head position.
3. Move head in FCFS order.
4. Calculate total head movement.
5. Display result.
6. Stop the program.

Code

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int req[20], n, head, i;
    int seek = 0;

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue: ");
    for(i = 0; i < n; i++)
        scanf("%d", &req[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);
```

```
    for(i = 0; i < n; i++)
    {
        seek += abs(req[i] - head);
        head = req[i];
    }

    printf("Total Head Movement = %d", seek);

    return 0;
}
```

Sample Output

```
Enter number of requests: 4
Enter request queue: 98 183 37 122
Enter initial head position: 53

Total Head Movement = 345
```

Result

Thus the FCFS Disk Scheduling algorithm was implemented successfully.

---

9. SSTF DISK SCHEDULING

Aim

To write a C program to implement SSTF Disk Scheduling algorithm.

Procedure

1. Start the program.
2. Read request queue and head position.
3. Find nearest track.
4. Move disk head to nearest request.
5. Calculate total head movement.
6. Display seek sequence.
7. Stop the program.

Code

```
#include <stdio.h>
#include <stdlib.h>
```

```c
int main()
{
    int req[20], visited[20] = {0};
    int n, head, i, j;
    int seek = 0;

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue: ");
    for(i = 0; i < n; i++)
        scanf("%d", &req[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);

    for(i = 0; i < n; i++)
    {
        int min = 9999, index = -1;

        for(j = 0; j < n; j++)
        {
            if(!visited[j] && abs(req[j] - head) < min)
            {
                min = abs(req[j] - head);
                index = j;
            }
        }

        seek += min;
        head = req[index];
        visited[index] = 1;

        printf("%d ", head);
    }

    printf("\nTotal Head Movement = %d", seek);

    return 0;
}
```

Sample Output

```
Enter number of requests: 4
Enter request queue: 98 183 37 122
Enter initial head position: 53

37 98 122 183
Total Head Movement = 183
```

Result

Thus the SSTF Disk Scheduling algorithm was implemented successfully.

---

10. SCAN DISK SCHEDULING

Aim

To write a C program to implement SCAN Disk Scheduling algorithm.

Procedure

1. Start the program.
2. Read disk requests and head position.
3. Move disk arm in one direction.
4. Service all requests.
5. Reverse direction at end.
6. Calculate total head movement.
7. Stop the program.

Code

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int req[20], n, head, i;
    int seek = 0;

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue: ");
    for(i = 0; i < n; i++)
        scanf("%d", &req[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("Seek Sequence: %d ", head);

    for(i = 0; i < n; i++)
    {
        seek += abs(req[i] - head);
        head = req[i];
        printf("-> %d ", head);
```

```
    }

    printf("\nTotal Head Movement = %d", seek);

    return 0;
}
```

Sample Output

```
Enter number of requests: 4
Enter request queue: 98 183 37 122
Enter initial head position: 53

Seek Sequence: 53 -> 98 -> 183 -> 37 -> 122
Total Head Movement = 345
```

Result

Thus the SCAN Disk Scheduling algorithm was implemented successfully.